

Trading RL System Design

Internship Screening Assignment - Written Response

Question 1 - Problem Decomposition

Six months sounds like a lot until you're two weeks in and still debugging your data pipeline. So I've split this into five milestones, each one producing something concrete that the junior engineer can look at and say 'yes, this is done.' No milestone ends with 'the model is training' - that's not a deliverable, that's a state.

Milestone	Deliverable	Success Criterion	Biggest Risk
M1 · Weeks 1-3 Data & Baseline	A clean OHLCV data pipeline plus a simple rule-based strategy (RSI mean-reversion) as a dumb benchmark to beat.	Pipeline produces gap-free, adjusted bars. The benchmark hits Sharpe ≥ 0.5 on held-out 2023 data. If we can't beat a moving average later, something is wrong.	Data quality. Missing bars, exchange halts, and survivorship bias are boring to fix but will silently corrupt everything built on top.
M2 · Weeks 4-7 Pattern Encoder	A trained contrastive encoder that maps any 60-bar OHLCV window to a 64-dim embedding - plus a clustering report showing what it found.	Embeddings show statistically different forward-return distributions across clusters (KS test, $p < 0.05$), and UMAP plots show at least 4 visually coherent groups.	The clusters turn out to reflect time-of-day or calendar effects rather than genuine price structure. This would make the whole encoder essentially useless.

Milestone	Deliverable	Success Criterion	Biggest Risk
M3 · Weeks 8-13 RL Agent (Sim)	A hierarchical RL agent - entry agent plus sizing agent - trained in a realistic backtest environment with transaction costs included.	In-sample: Sharpe $\geq$ 1.0, average RR $\geq$ 2.5, win rate $\geq$ 40%. Agent doesn't hold positions overnight more than 20% of trades.	Reward hacking. The agent discovers some degenerate strategy - like placing take-profits at $0.01 \times$ ATR and winning constantly on noise - instead of finding real setups.
M4 · Weeks 14-18 Walk- Forward Eval	A walk-forward backtest report across three out-of-sample windows, with a metric dashboard: Sharpe, Calmar, avg RR, win rate, max drawdown.	Sharpe $\geq$ 0.8 in at least 2 of 3 out-of-sample windows. Calmar $\geq$ 1.0. No single losing month exceeds 8% drawdown.	Overfitting. The agent may have memorised specific price sequences from the training period rather than learning anything transferable.
M5 · Weeks 19-24 Paper Trade + Deploy	A live paper-trading loop with a real broker feed, plus a position tracker and daily PnL report. Then controlled \$50 live deployment.	Paper-trade Sharpe stays within 15% of backtest Sharpe over 4 weeks. No execution errors. Average slippage below 0.1%.	Execution slippage hitting tight stop-losses harder than expected. A 0.05% slippage on a $0.5 \times$ ATR stop can meaningfully change the realised RR.

Question 2 - Pattern Learning Design

(a) What are we actually modelling?

The goal here is to answer a simple question: does this 60-bar window of market activity look like anything we've seen before? Not in a predictive sense - we're not asking 'will it go up?' We're asking 'what kind of moment is this?'

To do that, I take a sliding window of 60 bars and compute five features per bar

- all normalised so the model sees shapes rather than raw price levels:

- Log returns: $\log(\text{close} / \text{prev_close})$ - stationary and scale-invariant across assets
- Bar body: $(\text{close} - \text{open}) / \text{ATR}(14)$ - captures whether this was a decisive bar or indecisive
- Upper and lower wick ratios: each divided by $\text{ATR}(14)$ - tells us about rejection and absorption
- Volume ratio: $\text{bar volume} / 20\text{-bar rolling mean}$ - relative activity, not absolute numbers
- High-low range: $(\text{high} - \text{low}) / \text{ATR}(14)$ - how volatile was this specific bar

That gives a (60, 5) tensor per sample. The model is a Temporal Convolutional Network - a TCN - that outputs a 64-dimensional embedding. I chose a TCN over an LSTM because TCNs train faster and are more stable with the kind of noisy financial time series we're working with. The 64 dimensions are enough to represent meaningful variation without making downstream clustering hopeless.

To train it, I use contrastive learning - specifically a SimCLR-style setup. The idea is: take the same 60-bar window, create two slightly different views of it (one with mild Gaussian noise on volume, one with a small time-shift of ± 2 bars), and train the encoder to map both views to nearby points in embedding space, while pushing other windows away. The model learns what's stable about a window's structure - and that's exactly what a 'pattern' is.

Rough pseudocode:

```
encoder = TCN(input_dim=5, hidden_dim=128, output_dim=64, layers=4)
projection = MLP(64 -> 32)
```

```
for batch in dataloader:
    view_a = augment(batch) # mild volume noise
    view_b = augment(batch) # slight time shift
    z_a = projection(encoder(view_a))
    z_b = projection(encoder(view_b))
    loss = NT_Xent(z_a, z_b, temperature=0.1)
    loss.backward()
```

After training, the projection head gets thrown away. The raw 64-dim TCN output is what feeds into both the clustering analysis and, later, the RL observation space.

(b) Why this approach - and why not the others?

I went with self-supervised contrastive learning, and I want to be honest about why I rejected the alternatives rather than just advocating for my choice.

Supervised learning is off the table because it needs labels. You could proxy-label windows by future return direction, but now you've leaked future informa-

tion into the encoder. It's no longer discovering patterns - it's learning what was profitable in the past, which is a very different and much harder-to-generalise thing.

Pure unsupervised clustering (k-means or GMM directly on the raw features) is tempting because it's simple, but it has a core problem: distance in raw OHLCV feature space doesn't mean anything semantically. Two bars can be numerically close but structurally completely different. And you have to pick k upfront, with no principled way to do it.

Rule-based feature engineering - things like RSI, MACD, Bollinger Bands - is genuinely useful and I'd include it in the RL observation space. But it's not discovery. Every pattern you can find this way is a pattern someone already believed in. The whole point of the unsupervised phase is to find structure we didn't know to look for.

Self-supervised learning threads the needle: it doesn't need labels, it learns a meaningful embedding space, and by training on structural consistency across augmentations it discovers genuine recurring patterns rather than calendar effects or trivial correlations.

(c) How do we know it learned something before plugging it into RL?

This is the check I'd be most nervous about skipping. If we plug a garbage encoder into the RL agent, the agent will train for weeks on noise and we won't know why it isn't working. So before touching the RL, I'd run four checks:

- **Visual cluster inspection:** Run UMAP on the 64-dim embeddings, then HDBSCAN to get cluster labels. Plot them and look. Do the clusters correspond to anything recognisable - trending periods, choppy consolidation, volatility spikes, low-range compression? If everything is one big blob, the encoder collapsed.
- **Forward-return KS test:** For each cluster, compute the 10-bar forward log-return distribution. Run a Kolmogorov-Smirnov test between clusters. If the distributions are statistically different ($p < 0.05$), the embeddings capture something real. This isn't because we trained on forward returns - we didn't - it's a validation that the structure the model found is predictively meaningful.
- **Temporal confound check:** Verify that cluster membership isn't just a proxy for 'which year is this' or 'is VIX high.' If cluster 1 is entirely 2020 and cluster 2 is entirely 2023, we have a time-based confound, not a pattern. I'd run a simple logistic regression of cluster label $\sim$ year + month and check whether it's predictive.
- **Reconstruction sanity (optional):** Train a lightweight decoder on the frozen encoder and check how well it reconstructs the input window. If the bottleneck embedding can reconstruct the input at low error, we know the encoder hasn't collapsed to a constant. This is a quick 10-minute sanity check, not a core validation.

Question 3 - RL Action Space and Reward Design

(a) The action space

I'm using a hierarchical setup - two agents with different jobs. The reason is pretty intuitive: 'should I trade right now?' and 'where do I put my levels?' are meaningfully different decisions that benefit from being learned separately. Bundling them into one giant action space makes the learning problem unnecessarily hard, and it makes debugging nearly impossible.

Entry Agent - discrete, 4 actions:

- Hold - do nothing, or keep the current position
- Long - enter a long trade now
- Short - enter a short trade now
- Close - exit whatever position is open

Discrete makes sense here because the decision is categorical. You're either entering or you're not. A continuous 'confidence' value would just add noise.

Sizing Agent - continuous, 2 values (only fires after entry):

- SL distance: `a_sl` in $[0.3, 2.0]$ multiples of `ATR(14)`. So 0.5 means place your stop-loss at $0.5 \times \text{ATR}$ from entry.
- TP distance: `a_tp` in $[1.5, 6.0]$ multiples of `ATR(14)`. So 3.0 means take-profit sits at $3 \times \text{ATR}$ from entry.

Everything is expressed in ATR multiples so the agent learns distances that scale correctly across different volatility environments. A 10-pip stop on a calm day and a 10-pip stop during a news spike are completely different risks - ATR normalisation handles that automatically. The TP minimum of $1.5 \times \text{ATR}$ ensures RR is never degenerate even at the widest stop.

(b) The reward function

The reward is calculated at trade close, not during the trade. I made this choice deliberately - if you give intermediate rewards, the agent learns to close early for small wins and avoid the larger-but-uncertain TP targets. That's the exact opposite of what we want.

Formula:

$$\begin{aligned} R = & \text{PnL_norm} \\ & + 0.3 \times \text{RR_bonus} \\ & - 0.5 \times \text{SL_penalty} \\ & - 0.01 \times \text{bars_held} \\ & - 0.2 \times \text{overtrade_flag} \end{aligned}$$

Let me walk through what each term actually does and why it's there:

- **PnL_norm**: Realised profit divided by the SL distance. A winning trade that hits TP gives +RR as a reward; a losing trade gives -1. This makes every trade comparable regardless of how far the levels were placed. It

also means the agent is rewarded more for placing ambitious TPs that actually get hit.

- **RR_bonus (weight 0.3)**: This kicks in only on winning trades: $0.3 \times (a_tp / a_sl - 2.0)$. If RR is exactly 2, the bonus is zero. If RR is 3, the bonus is 0.3. It's a gentle nudge toward ambitious targets - present but not dominant, so the agent doesn't sacrifice win rate chasing unreachable TPs.
- **SL_penalty (weight 0.5)**: Equal to $0.5 \times \max(0, a_sl - 1.0)$. If the agent places a stop tighter than $1 \times \text{ATR}$, no penalty. Wider than $1 \times \text{ATR}$, it pays proportionally. This is how we enforce 'tight stop-losses' without making it a hard constraint that the agent games.
- **Hold penalty (0.01 per bar)**: A small per-bar cost while holding a position. This discourages the agent from holding indefinitely hoping TP eventually gets hit on a slow market. It also implicitly rewards decisive entries.
- **Overtrade flag (0.2)**: Applied whenever the agent opens a trade within 5 bars of the last close. This is the mechanism that teaches patience - entering again immediately usually means chasing, not finding a quality setup.

(c) The reward hack I'm most worried about - and how to prevent it

The hack I'd expect to see first: the agent figures out that if it sets TP incredibly tight - say $0.05 \times \text{ATR}$ - it wins almost every single trade. Price moves by that amount within a bar or two basically by random noise. The PnL_norm goes positive very often, and because RR is below 2 the RR_bonus doesn't fire. But the agent is still accumulating positive reward by winning constantly at tiny margins. It's discovered micro-scalping as a degenerate solution.

How I'd prevent it:

- Hard floor on TP in the action space: $a_tp \geq 1.5 \times \text{ATR}$. This is enforced in the environment itself, not the reward. The agent literally cannot set a TP below this. Hard constraints beat soft penalties when the behaviour is truly unacceptable.
- The overtrade penalty makes rapid trading expensive. A micro-scalping strategy requires many trades in quick succession, and each one inside the 5-bar window gets hit with the 0.2 penalty. It accumulates fast.
- Track realised RR as a logged metric during training - separate from the reward. If it drops below 1.5 over any 1000-trade window, that's a signal to investigate. This catches hacks that slip through the reward before they waste weeks of compute.

Question 4 - Evaluation and Deployment

(a) How would I evaluate before going live?

The evaluation only counts if it happens on data the agent never saw - not just during training, but also not during hyperparameter tuning. I'd run three

separate out-of-sample windows and require the agent to meet the thresholds in at least two of them.

Metric	What it measures	Threshold	Why that number
Sharpe Ratio (annualised)	Return per unit of risk, annualised. The most standard single-number summary of risk-adjusted performance.	≥ 1.0 out-of-sample	At Sharpe 1.0 you're earning one unit of return per unit of risk. Below this, realistic slippage will likely push you negative.
Calmar Ratio	Annualised return divided by the worst drawdown. Captures whether the return justifies the pain of holding through losing streaks.	≥ 1.0	Calmar < 1 means the worst drawdown exceeded your annual return. That's not a strategy worth running.
Average Realised RR	Mean of (TP distance / SL distance) across all closed trades. Direct check that the agent is doing what it was designed for.	≥ 2.0	This is the core design goal. If average RR is below 2, the agent isn't finding high-quality setups - it's just trading.
Win Rate	Percentage of trades that hit TP rather than SL.	$\geq 35\%$	At RR 2.0, a 35% win rate gives positive expectancy: $(0.35 \times 2) - (0.65 \times 1) = +0.05$. Below 35% at that RR, the system is unprofitable.
Max Draw-down	Largest peak-to-trough decline in the equity curve.	$< 15\%$	On a \$50 account this is \$7.50. Beyond this the system's variance is too high to trust for scale-up, and also just uncomfortable to watch.
Trade Frequency	Average trades per week. Not a performance metric - a sanity check.	2-10 per week	Fewer than 2 and we can't measure anything statistically. More than suggests the agent is overtrading, which contradicts the whole premise.

One thing these metrics don't capture: the agent's behaviour in regimes it's never seen. A flash crash, a major macro event, a liquidity crisis - the metrics above look fine right up until they don't. That's a known gap.

(b) Three things I'd do the day before going live that I wouldn't have bothered with during development

1. Rerun the entire backtest with simulated slippage

During development I almost certainly assumed perfect fills at the bar close. That's fine for iteration, but it's not real. The day before going live I'd rerun the full out-of-sample backtest with 0.05% slippage per fill - which is a reasonable conservative estimate for a small retail account. If Sharpe drops below 0.7, the strategy is too fragile to run. Tight stop-losses are especially vulnerable here: a fill that's 0.05% worse than expected can meaningfully shift the realised RR on a trade with a 0.5x ATR stop.

2. Build and test a kill switch

During development, a bad run just means a wasted training cycle. In live trading, a bug can open a position and never close it. The day before going live I'd write a monitoring script that polls the broker API every 60 seconds, logs all open positions and their current PnL, and automatically closes everything flat if intraday drawdown hits \$10 (20% of the \$50 account). Then I'd test it - manually open a paper position and confirm the kill switch actually fires. This is the kind of thing you'd never bother testing during development because the stakes aren't real yet.

3. Walk through one complete trade cycle and verify every timestamp

A surprising number of live trading bugs come from timezone mismatches. The model trained on UTC data. The broker delivers timestamps in EST. The conversion breaks at a DST transition and suddenly the agent is trading on stale bars without knowing it. The day before going live I'd manually trace one complete trade - bar received, observation constructed, action produced, order placed, fill confirmed - and log the timestamp at every step. I'd verify the data the agent sees in paper-trade mode is byte-for-byte identical to what it would have seen during training on the same bars. Boring to check. Critical to get right.

Question 5 - Self-Critique

Honestly, this is the question I find most useful. It's easy to write a design that sounds coherent. It's harder to identify where it's actually fragile. Here are the three places I'd push back on my own answers.

(a) The three weakest decisions

Weak Decision 1: The 60-bar window is arbitrary

I picked 60 bars because it felt intuitive - roughly one trading session for 5-minute data. But there's no principled reason the patterns we care about manifest at that horizon. A breakout might be building over 200 bars. A key rejection

candle might be meaningful over 3. Forcing a single fixed window means the encoder tries to find all structure at one timescale, which is almost certainly wrong.

When it would be right: If we had prior evidence - from autocorrelation analysis of the specific asset - that dominant regime cycles have periods close to 60 bars, and if the RL agent's decision horizon matched that same scale. In practice I'd experiment with a multi-scale encoder (20-bar, 60-bar, 200-bar in parallel) but kept this simple to stay implementable for a junior.

Weak Decision 2: The 5-bar cooldown is a blunt instrument

Penalising any trade within 5 bars of the last close doesn't adapt to the market. On a strong trending day there might be three genuinely good sequential entries. On a choppy ranging day, no trade within 50 bars might be sensible. A fixed cooldown doesn't know the difference. Worse, the agent might game it by mechanically waiting 6 bars regardless of market context.

When it would be right: If the asset has a predictable mean-reversion cycle - say a futures contract that characteristically ranges within a session before resetting - then a fixed cooldown approximately matches the natural rhythm. For trending assets, this penalty would actively hurt performance.

Weak Decision 3: The encoder validation doesn't actually test what matters

The mutual information / KS test I proposed checks whether cluster labels predict forward returns. That's a reasonable sanity check, but it's not the right question. The encoder's job is to give the RL agent useful context for making decisions - not to predict returns directly. A cluster might have low MI with 10-bar forward returns and still be incredibly useful to the agent because it captures risk structure, not directional signal. Conversely, a cluster with high MI might be useless at the agent's actual decision timescale.

When it would be right: If the RL agent's decision horizon closely matched the forward return window used in the test, and if the embedding's primary job was return-direction prediction. A better validation would be training a supervised probe on the embeddings to classify manually labelled regimes - but that requires labelled data, which is exactly what I was trying to avoid.

(b) The question I wish had been asked

"What happens when the market does something the agent has never seen before?"

This is the most important practical problem in live trading systems and the assignment didn't ask it directly. The RL agent is trained on historical data and will encounter distribution shift the moment it goes live - because markets evolve, participants change, and macro regimes shift. A genuine structural break - a flash crash, a central bank surprise, a liquidity crisis - could cause the agent to behave in ways that are unpredictable and potentially expensive.

My answer would focus on three things:

- Build an out-of-distribution detector that runs in production. If the current embedding falls far from any cluster centroid seen during training - measured by Mahalanobis distance - the system goes flat and stays flat until the regime stabilises. 'I don't recognise this market' is a valid and profitable output.
- Maintain a replay buffer of recent live data and fine-tune the encoder and agent weekly, weighting recent bars more heavily. This handles slow drift without requiring a full retraining cycle.
- Accept that there will be periods of being out of the market entirely. The alternative - trading confidently in a regime the model has never seen - is how these systems blow up. Sitting flat is not a failure.